

Empirical Evaluation of a Validator-Assisted LLM NetOps Pipeline: A Preregistered Paired Study with gpt-5-mini on 400 Intent→Configuration Tasks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered (CAND-2026-002) and executed a paired experiment to measure whether a lightweight deterministic syntax/policy validator (with at most one bounded LLM repair) reduces operational failures when applying LLM-generated network changes. Using gpt-5-mini via a hosted NetOps adapter on 400 public/synthetic intents (shared_initial_candidate semantics), we ran 800 emulator episodes with full archival of provider calls, validator logs, scorer JSONs, and SHA-256s. All paired episodes were operationally successful under both arms (n11=400, n10=0, n01=0, n00=0). The paired difference (Guarded - Baseline) = 0.00 (95% bootstrap percentile CI 0.00–0.00, 10k resamples, seed=1729); McNemar exact two-sided p=1.0. Crucially, because generation_semantics was preregistered as shared_initial_candidate and the deterministic validator accepted every identical shared candidate (validator_reject_count=0, repair_calls=0), the Guarded intervention was empirically inert on this task bank. We present this run as a reproducible boundary case, release full artifacts (execution/raw_results.jsonl SHA-256 871b2978...), and derive concrete, artifact-grounded recommendations for future preregistered evaluations of validator efficacy in NetOps pipelines.

I. INTRODUCTION

Automating intent→configuration translation with LLMs promises faster NetOps workflows but raises safety and correctness concerns: a single incorrect change can produce reachability loss, policy violation, or extended outage. A common mitigation is generate→validate→repair: produce candidate configs with an LLM, run an automated validator (syntax, policy checks), and optionally repair with a bounded LLM call. Prior work advocates verifier-guided repair and RAG/verifier loops, but reproducible, preregistered measures of validator efficacy in paired experimental designs are limited. We preregistered and executed CAND-2026-002 to answer: for a paired sample of intents, how much does a deterministic syntax/policy validator (with ≤1 bounded repair) reduce operational failure proportion vs direct application? Contributions. (1) A fully preregistered, artifact-hashed paired experiment (400 intents, 800 episodes) with complete archival of provider calls, validation logs, and scorer outputs. (2) A concrete, reproducible boundary outcome demonstrating how generation semantics can make a Guarded intervention empirically inert. (3) Artifact-grounded prescriptive recommendations and an explicit mapping from principal reported numbers to archived artifacts and SHA-256s.

II. RELATED WORK

We positioned this study against recent NetOps/AIOps and generate-validate-repair literature, verified and used to motivate metrics and validator design: a NetOps/AIOps survey and architecture synthesis (OpenAlex W4415071524), intent→configuration LLM translation (doi:10.1002/nem.2313), verifier-guided IaC repair studies (doi:10.2139/ssrn.6608518; OpenAlex W7124141852), and continuous network compliance/verification tools (OpenAlex W4401857375). We explicitly do not claim general superiority or production readiness for any validator design; instead we test a narrowly preregistered hypothesis under tightly controlled semantics and release all artifacts for community audit.

III. METHODOLOGY

Preregistration. The experiment was preregistered and frozen as CAND-2026-002 prior to any model calls. Key fixed choices: paired binary design; 400 unique intents executed as paired episodes (Baseline vs Guarded); model=gpt-5-mini via adapter family hosted_netops_gvr_v1; generation_semantics=shared_initial_candidate (a single shared candidate produced once and reused by both arms); validator = deterministic syntax/policy checker; repair policy = at most one deterministic LLM repair call if validator rejects; maximum_model_calls=800; analysis = McNemar exact two-sided test with 95% bootstrap percentile CI (10k resamples, seed=1729). The preregistration object and all parameters are archived in execution_manifest (study_id CAND-2026-002). Execution protocol. For each intent the pipeline produced one shared candidate configuration (file: execution/responses/task-<id>-shared-initial.txt; SHA-256s recorded in execution_manifest). Baseline applied that shared candidate directly to an emulator replica; Guarded first ran the deterministic validator on the same shared candidate and would invoke one deterministic repair call only on validator rejection. All provider calls, transformation manifests, and execution logs were archived (paths listed in execution_manifest). Scoring. Operational failure = emulator flag for reachability loss or policy violation; per-task scorer JSONs saved under execution/scoring/task-<id>-.json (SHA-256s recorded). Reproducibility pointers (principal): raw results execution/raw_results.jsonl SHA-256 871b297808ae2bb6b26260c99299d5c493afd3b03232ca4d92c39ca5ffaab86

paired contingency table analysis/paired_contingency_table.csv SHA-256 fad28887d8b3847baaebe368d178ba14986eae633d789dae1396b3614b1d
 paired difference figure analysis/paired_difference_figure.svg SHA-256 ae5b8e0c644f8cf7579ff0b2daec77475310d4de96da6a4f1a9266a64b724e4e
 All per-call caches are available under execution/call_cache/ with their SHA-256s (see execution_manifest).

IV. RESULTS

Execution and confirmatory results (artifact-grounded). completed_episode_count=800; completed_pair_count=400; model_calls_used=400; validator_reject_count=0; deterministic_repair_calls_invoked=0. The 2x2 paired contingency (analysis/paired_contingency_table.csv, SHA-256 fad28887...) is: n11=400 (both arms success), n10=0 (Baseline success only), n01=0 (Guarded success only), n00=0 (both arms failure). From archived scorer JSONs: Baseline success rate = 400/400 = 1.000; Guarded success rate = 400/400 = 1.000. Paired difference (Guarded - Baseline) = 0.00. Bootstrap (10,000 resamples, seed=1729) 95% percentile CI = [0.00,0.00] (artifact: analysis/paired_difference_figure.svg SHA-256 ae5b8e0c...). McNemar exact two-sided p = 1.0. Diagnostic finding. Because generation_semantics was shared_initial_candidate and the deterministic validator accepted every identical shared candidate (validator_reject_count=0), no repair calls were triggered and Guarded was not exercised; therefore the experiment produced a reproducible boundary case rather than an informative test of validator efficacy for the sampled tasks. Representative artifact trace (task-000001): execution/responses/task-000001-shared-initial.txt SHA-256 f8de4d6e1a625e42...; execution/scoring/task-000001-baseline.json SHA-256 ffb47774de57...; execution/scoring/task-000001-guarded.json SHA-256 9e70f3a4b18d... (all per-task hashes enumerated in execution_manifest).

V. DISCUSSION

Interpretation. The numerical analysis is correct and fully reproducible from archived artifacts: under the preregistered shared_initial_candidate semantics the validator never rejected a candidate and thus could not reduce operational failures. This is an important boundary outcome: a null paired difference in this run is a consequence of the interaction between generation semantics and validator decisions, not evidence that validators are unnecessary in general. Artifact-grounded recommendations for future preregistered validator evaluations (all are supported by archived evidence and the observed inert intervention): 1) Use independent_condition_generation=true for paired designs so each arm receives independently generated candidates and the validator can be exercised; 2) Include preregistered positive controls (small set of intents crafted to elicit invalid candidates) to verify validator liveness and scorer sensitivity; 3) Ensure task sampling includes historically difficult change types (ACLs, routing) or adversarial templates to avoid a passive task bank; 4) Publish validator rule set and decision

logs (we archived execution/transformation_manifest.json and execution/execution_log.jsonl) so auditors can inspect why reject occurred or did not occur; 5) Preserve provider_call caches (we archived execution/call_cache/) to enable exploratory replay with alternative generation semantics without external dependency. Threats to validity (artifact-grounded). Internal: single model (gpt-5-mini), single adapter, and shared generation semantics limit the internal generality. Construct: scorer/emulator fidelity drives detection; archived scorer JSONs exist but emulator realism remains limited vs production. External: synthetic/public task bank and emulated execution limit generalization. Statistical note. The degenerate 2x2 table (all concordant successes) yields CI=[0,0] and p=1.0; these are mathematically correct but uninformative for validator efficacy and must be interpreted as a design-driven boundary case. Reviewer requests addressed. We: (a) reframed the run as a reproducible boundary case; (b) provided explicit artifact pointers and SHA-256s for every principal number; (c) included the validator decision logs and count (validator_reject_count=0) so readers can audit why the intervention was inert; (d) recorded concrete prescriptive changes for future preregistered work.

VI. LIMITATIONS

- Single model and single adapter (gpt-5-mini via hosted_netops_gvr_v1); results do not generalize to other LLMs, adapters, or model families.
- Preregistered generation_semantics = shared_initial_candidate: because the validator accepted every identical shared candidate (validator_reject_count=0), the Guarded condition was not exercised and the confirmatory hypotheses about validator efficacy could not be tested on these data.
- Public/synthetic intent bank and emulated execution limit realism vs production traces and hardware; scorer behavior depends on emulator fidelity.
- No injected positive controls were present in this run; the archived artifacts alone cannot demonstrate scorer/validator sensitivity to adversarial or pathological inputs without new execution.
- External validity limited: vendor/device diversity, adversarial prompts, and long-term deployment dynamics are not covered by this run.

VII. CONCLUSION

A rigorously preregistered and fully archived paired run with gpt-5-mini over 400 intents produced complete concordant successes under both Baseline and Guarded arms because the shared_initial_candidate generation semantics produced identical candidates that the deterministic validator accepted (validator_reject_count=0). The run therefore documents a reproducible boundary case and demonstrates the critical role of generation semantics and task sampling in testing validator efficacy. We release full artifacts (execution/raw_results.jsonl SHA-256 871b2978...) so others can reproduce this outcome,

audit validator rules and logs, and rerun alternative preregistered designs (e.g., independent generation or injected failing controls) to obtain informative estimates of validator effect sizes.

DISCLOSURE STATEMENT

Mandatory Disclosure (artifact-grounded). Human involvement and autonomy boundary: after an autonomy lock the autonomous system performed literature review, candidate study selection, preregistration (CAND-2026-002), code generation, experiment execution, analysis, peer-review response, and manuscript drafting and revision. The Research Director selected the topic family (Generative AI / LLMs for NetOps) and approved pre-lock infrastructure development; no human manually edited or rewrote technical manuscript content after the autonomy lock. Models and adapter: model=gpt-5-mini (calls via hosted NetOps adapter family hosted_netops_gvr_v1). Exact master prompts, system instructions, and generation templates used for each shared_initial candidate are archived in execution/raw_results.jsonl and in the provider call cache files recorded in execution_manifest. Preregistration/freeze: the preregistration (study_id=CAND-2026-002) was frozen before any model calls and specified the research question, confirmatory hypotheses (H1–H4), primary estimand (paired difference), generation_semantics=shared_initial_candidate, analysis plan (McNemar exact; bootstrap CI 10k resamples, seed=1729), sample size (400 paired intents), maximum_model_calls=800, retry policy, and stopping rules; the full preregistration object is archived in execution_manifest.preregistration. Framework and validator: experiments used hosted_netops_gvr_v1 to call gpt-5-mini; the deterministic syntax/policy validator and the bounded repair template are archived (execution/transformation_manifest.json and execution/execution_log.jsonl). Execution manifest and hashes: execution/raw_results.jsonl SHA-256 871b297808ae2bb6b26260c99299d5c493afd3b03232ca4d92c39ca5ffaab865; analysis/paired_contingency_table.csv SHA-256 fad2887d8b3847baaebe368d178ba14986eae633d789dae1396bce3bf14abf; representative response: execution/responses/task-000001-shared-initial.txt SHA-256 f8de4d6e1a625e42ba4907440b4377c78949ed56446e5c73030bc267e56abfaf; scorer JSON: execution/scoring/task-000001-baseline.json SHA-256 ffb47774de570d1f8417a3191cca87aac22ddcda4d246f3224e8ef6284c177. Technical restarts and failures: started_at_utc and completed_at_utc are recorded in execution_manifest; completed_episode_count=800; failed_episode_count=0; model_calls_used=400. Pre-lock development: pre-lock infrastructure development and rehearsal occurred but were explicitly excluded from final empirical evidence; only post-lock archived artifacts reported here serve as evidence. Key interpretive caveat required by the run: the preregistered generation_semantics was shared_initial_candidate and the deterministic validator accepted every shared candidate (validator_reject_count=0; repair_calls=0). Therefore the

Guarded intervention was empirically inert on this task bank and adapter; this manuscript frames the run as a reproducible boundary case rather than evidence for or against validator utility in general. All archived files and SHA-256 hashes are released to enable exact reproduction and follow-on preregistered experiments.

REFERENCES

- [1] Jibum Hong, Jibum Hong, Nguyen Van Tu, James Won-Ki Hong, James Won-Ki Hong, "A Comprehensive Survey on LLM-Based Network Management and Operations", 2025, 10.1002/nem.70029
- [2] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518
- [4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Shahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026
- [5] Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, Qing Li, "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models", 2024, 10.1145/3637528.3671470
- [6] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, Laurent Callot, "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback", 2026, 10.1145/3786583.3786898